

# Information Assurance and Security Project

## Project 4 : Password Strength Tester

“ You can build a cyber security project to test the strength of your passwords according to the OWASP (Open Web Application Security Project) standards.

The deliverable should enforce basic rules like minimum password length, and complexity, avoiding parts of username or any public information, the inclusion of numerical and special characters, and other ground rules while providing user flexibility. The system can encourage using passphrases over a standard password, making it easier for users. Password strength testers should give a thumbs up only when all the test cases pass. ”

### **INTRODUCTION :**

In today’s digital world, passwords continue to be the most common method for protecting online accounts and sensitive data. They are the first line of defense against unauthorized access. However, weak or easy-to-guess passwords continue to be one of the leading causes of security breaches. Many users rely on simple, predictable passwords due to convenience, which makes them vulnerable to attacks such as **brute force, dictionary, or credential stuffing**.

This Password Strength Tester, aims to evaluate and improve password security in accordance with the OWASP (Open Web Application Security Project) guidelines. The system evaluates how secure a password is based on key parameters such as **minimum length, complexity, entropy, inclusion of numeric and special characters**, and the **avoidance of personal or publicly known information**. By doing so, it ensures that password creation meets standardized security criteria while **maintaining user flexibility**. It also encourages the use of **passphrases** (longer, memorable combination of words) which are both user-friendly and highly resistant to attacks. Another key feature is its ability to provide **immediate feedback** to the user. When a password is entered, the system evaluates its strength and clearly indicates whether it is weak, moderate, or strong. It can also highlight specific weaknesses and suggest improvements, helping users better understand what makes a password secure.

Overall, the Password Strength Tester serves as an educational and practical cybersecurity application that promotes better password hygiene and strengthens user authentication mechanisms in digital systems.

### **PART 1 : Main Concepts**

- **Password Authentication :**

In this process, the user creates a username and a password that are stored by the system. Then during login, the user enters their password and the system compares it with the one stored. If they match, it grants access, if not a denial message will appear.

The exact functional flow of password authentication typically follows these steps :

1. The user creates a username and password.
2. The system stores the password securely.
3. During login, the user enters the password.
4. The system compares the entered password with the stored one.
5. Access is either granted or denied.

The strength of the password is critical to security. Which is why, most systems will only accept passwords that meet all security requirements.

- Password Attacks :

Brute Force Attack : An attacker tries all possible combinations until the password is found.

Dictionary Attack : Attackers use a list of common passwords to guess the right one.

Credential Stuffing : Attackers reuse passwords stolen from data breaches on other websites.

Social Engineering : Attackers guess passwords based on personal information.

- Password Strength :

To reduce security breaches as much as possible, a password must meet specific requirements. A strong password has ; length, combination of characters and numbers, and no use of personal or publicly known information.

- Typical policies implemented by organizations include ;

Minimum length (8–12 characters)

Uppercase and lowercase letters

At least one number

At least one special character

Avoid common passwords

Avoid using username in the password

- Password Entropy :

The more random and unpredictable a password is, the higher entropy is.

Longer combinations are harder to guess like « MySecurepassW@987 » compared to « name123 ».

## **PART 2 : State of the Art ( Existing Solutions)**

Password strength testers are evaluation tools that companies use to help create secure passwords based on several criteria such as length, complexity, and predictability.

The most widely known ones are :

- zxcvbn Password Strength Estimator : (developed by Dropbox)

Estimates how long it would take to crack a password ;

Detects **common patterns**, dictionary words, and keyboard patterns (like qwerty).

Advantages : **very accurate** and widely used.

Disadvantages : requires a large dictionary database.

- Kaspersky Password Checker :

Tells if a password was found in known leaks, and gives security recommendations.

Advantages : very reliable, **checks breach** databases.

Disadvantages : requires internet connection.

- Bitwarden Password Strength Tool :

Evaluates password entropy, checks complexity rules, and generates secure passwords.

Advantages : integrated with a password manager, supports **password generation**.

### Limitations :

Although these tools are powerful, some of them can be complex, require large datasets, or are not easy to understand for beginners.

The objective of this project is to implement a simpler solution that follows the recommendations of the OWASP.

### **PART 3 : High Level Design**

- [System Overview :](#)

The Password Strength Tester is designed as a simple web-based application that evaluates the security level of a user's password based on predefined security rules.

The system analyzes the password entered by the user, checks it against a set of security rules, calculates a strength score, and provides feedback indicating whether the password is weak, medium, or strong. The goal is to help users understand the quality of their passwords and encourage stronger password creation.

- [System Architecture :](#)

#### 1/ User Interface UI :

- ➔ Accepts username and password input
- ➔ Encourages passphrase usage
- ➔ Displays feedback results

#### 2/ Input Validation Module :

- ➔ Checks that the input fields are not empty
- ➔ Checks the input format
- ➔ Sends the password to the next module

#### 3/ Password Analysis Module :

- ➔ Checks the password against all necessary security rules

Examples of rules checked:

Minimum password length

Presence of uppercase lowercase letters

Presence of numbers and special characters

Avoidance of username in the password

#### 4/ Strength Evaluation Engine :

- ➔ Calculates the password strength score

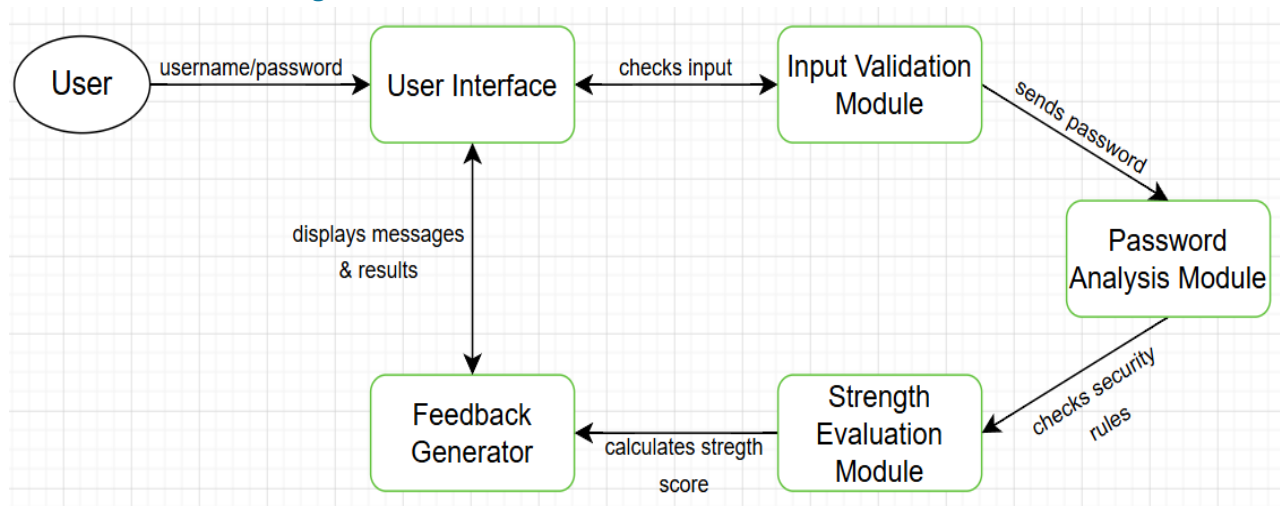
Length  $\geq 8$ , Uppercase/Lowcase Letter, Number, Special character = +1 for each

➤ 0-2 = weak                      3-4 = medium                      5+ = strong

#### 5/ Feedback Generator :

- ➔ Sends the strength score to the UI
- ➔ Sends messages to help improve password to UI

- [Architecture Diagram :](#)



- [Functional Flow :](#)

The functional flow of the system describes how the password evaluation process occurs:

1. The user enters a username and password in the interface.
2. The input validation module checks whether the fields are valid.
3. The password is sent to the password analysis module.
4. The system evaluates the password against security rules.
5. The strength evaluation engine calculates a strength score.
6. The feedback generator prepares the result and recommendations.
7. The results are displayed on the user interface.

- [Data exchanged :](#)

- |                          |                        |
|--------------------------|------------------------|
| 1) User → UI             | Username / Password    |
| 2) UI → Validation       | Input data             |
| 3) Validation → Analysis | Password               |
| 4) Analysis → Evaluation | Security rules results |
| 5) Evaluation → Feedback | Strength score         |
| 6) Feedback → UI         | Result / suggestions   |

#### **PART 4 : Tools and Development Phases**

- [Tools used :](#)

Programming languages : HTML, CSS, JavaScript.

Development Environment : Visual Studio Code.

Files Storage : GitHub.

- [Development Phases :](#)

- Requirement Analysis : identification of the objectives, concepts, and requirements of the project. The main goal is to determine the security rules that the password tester will follow.
- System Design : identification of the main components of the system (User Interface, validation module, analysis module, evaluation engine, and feedback generator) are identified, as well as the data flow between them.
- Implementation : development of the application and the password strength evaluation logic.
- Testing : using various passwords to verify that the system is working correctly.
- Deployment and Documentation